

南開大學
Nankai University

《计算机组成原理》指令集实验指导手册

数据分组统计：
ARM vs RISC-V 指令集对比实验

姓名：赵宇哲

学号：2411094

班号：1011

2026 年 4 月 30 日

目录

1 实验目的 **2**

2 实验背景与问题描述 **2**

3 算法设计 **3**
3.1 算法基本思路 . . . 3
3.2 算法伪代码 . . . 3
3.3 核心判断逻辑说明 . . . 3

4 实验环境 **4**

5 ARM 程序设计 **5**
5.1 程序结构 . . . 5
5.2 寄存器使用说明 . . . 5
5.3 核心代码说明 . . . 5

6 RISC-V 程序设计 **6**
6.1 程序结构 . . . 6
6.2 寄存器使用说明 . . . 6
6.3 核心代码说明 . . . 7

7 实验步骤 **7**
7.1 编译 ARM 程序 . . . 7
7.2 运行 ARM 程序 . . . 7
7.3 编译 RISC-V 程序 . . . 8
7.4 运行 RISC-V 程序 . . . 8
7.5 验证计数结果 . . . 8
7.6 反汇编查看程序结构 . . . 9
7.7 观察与记录重点 . . . 9

8 预期结果与分析引导 **10**

9 实验流程设计 **12**
9.1 实验整体流程 . . . 12
9.2 算法执行流程 . . . 13
9.3 算法伪代码 . . . 14

10 实验评价方法 **15**

11 实验总结 **15**

1 实验目的

本实验围绕大数据处理中的数据分组统计问题，设计并实现一个简单的阈值统计实验。实验给定一组整数数据和一个阈值，要求程序依次遍历数组，统计其中大于等于阈值的数据个数，并计算这些数据的总和。

本实验主要验证 ARM 与 RISC-V 在条件判断实现方式上的差异。对于同一个条件统计逻辑，ARM 程序采用条件执行指令完成条件计数和条件累加；RISC-V 程序则通过显式条件分支指令控制是否执行统计操作。

通过本实验，希望达到以下目标：一是理解简单数据统计问题在汇编层面的实现过程；二是能够通过 QEMU 用户态模拟器运行 ARM 和 RISC-V 汇编程序；三是能够结合反汇编结果，观察并分析两种指令集在条件判断和控制流实现方式上的不同。

2 实验背景与问题描述

在大数据处理场景中，数据筛选、计数和求和是十分常见的基础操作。例如，电商平台可能需要从一批订单金额中筛选出大于某个阈值的高价值订单，并统计这些订单的数量和总金额；日志分析系统也可能需要统计访问量超过某个标准的记录。虽然这类问题本身并不复杂，但它能体现计算机处理批量数据时常见的循环、访存、比较和条件判断过程。

为了便于在汇编层面实现和观察，本实验将实际问题简化为一个整数数组的阈值分组统计问题。实验中给定 8 个整数数据，设置阈值为 10。程序需要从数组首元素开始依次读取数据，如果当前数据大于等于 10，则将计数器加一，并把该数据累加到总和中；如果当前数据小于 10，则不进行统计，直接进入下一轮循环。

本实验使用的数据如下：

```
8, 12, 6, 20, 15, 9, 3, 11
```

阈值设置为：

```
10
```

其中，大于等于阈值的数据为：

```
12, 20, 15, 11
```

因此，满足条件的数据个数和求和结果分别为：

```
count = 4
sum = 12 + 20 + 15 + 11 = 58
```

选择该问题作为实验内容，是因为它的算法逻辑比较清晰，汇编实现难度适中，同时又包含了典型的条件判断结构。特别是其中的 `if` 判断部分，可以较好地体现 ARM 条件执行指令和 RISC-V 显式分支指令之间的差异。

3 算法设计

3.1 算法基本思路

本实验采用顺序遍历数组的方法完成数据分组统计。程序首先设置数组首地址、数组长度和阈值，并将计数器与求和变量初始化为 0。随后进入循环，每次从数组中读取一个整数，将其与阈值进行比较。

如果当前数据大于等于阈值，则说明该数据属于需要统计的分组，此时执行两步操作：第一步将计数器加一，第二步将当前数据累加到总和变量中。如果当前数据小于阈值，则跳过这两步统计操作，继续处理下一个数组元素。

在本实验中，ARM 和 RISC-V 两个程序实现的算法功能完全相同，区别主要体现在条件判断之后的控制方式上。ARM 使用带条件后缀的指令完成条件更新，而 RISC-V 使用条件分支指令决定是否跳过统计代码。

3.2 算法伪代码

本实验对应的伪代码如下：

```
data = {8, 12, 6, 20, 15, 9, 3, 11};
threshold = 10;
count = 0;
sum = 0;

for (i = 0; i < 8; i++) {
    if (data[i] >= threshold) {
        count++;
        sum += data[i];
    }
}
```

其中，`data` 表示待处理的数据数组，`threshold` 表示阈值，`count` 用于记录满足条件的数据个数，`sum` 用于记录满足条件的数据总和。

3.3 核心判断逻辑说明

本实验的核心判断逻辑如下：

```
if (data[i] >= threshold) {
    count++;
    sum += data[i];
}
```

这段逻辑在高级语言中表现为一条简单的 `if` 语句，但在汇编语言中需要通过比较指令和条件控制机制实现。ARM 程序中，先使用 `cmp` 指令比较当前数据和阈值，然后使用 `addge` 指令在条件成立时完成计数和求和。RISC-V 程序中，则使用 `blt` 指令判断当前数据是否小于阈值，如果条件成立就跳转到 `skip` 标签，从而跳过计数和求和指令。

因此，本实验观察的重点不是算法复杂度，而是同一个条件统计逻辑在 ARM 和 RISC-V 两种指令集中的不同实现方式。

4 实验环境

本实验在 Ubuntu 22.04 LTS 的 WSL2 环境下完成，使用 QEMU 用户态模拟器运行 ARM 和 RISC-V 汇编程序。根据实验要求，本实验使用的 QEMU User 版本为 6.2.0，其中 ARM 程序使用 `qemu-arm` 运行，RISC-V 程序使用 `qemu-riscv64` 运行。

实验中使用的主要工具如下：

- 操作系统环境：Ubuntu 22.04 LTS；
- 编辑工具：Visual Studio Code；
- QEMU 用户态模拟器：QEMU User 6.2.0；
- ARM 汇编工具：`arm-linux-gnueabi-as`；
- ARM 链接工具：`arm-linux-gnueabi-ld`；
- ARM 运行工具：`qemu-arm`；
- RISC-V 汇编工具：`riscv64-linux-gnu-as`；
- RISC-V 链接工具：`riscv64-linux-gnu-ld`；
- RISC-V 运行工具：`qemu-riscv64`；
- 反汇编工具：`arm-linux-gnueabi-objdump` 和 `riscv64-linux-gnu-objdump`。

在 Ubuntu 中，可以使用以下命令安装实验所需工具：

```
sudo apt update
sudo apt install qemu-user gcc-arm-linux-gnueabi \
binutils-arm-linux-gnueabi gcc-riscv64-linux-gnu \
binutils-riscv64-linux-gnu
```

本实验中的 QEMU User 版本信息如图 4.1 所示。

```
eugene@Aurora:/mnt/c/Users/19902/zhilingji_experiment$ qemu-arm --version
qemu-arm version 6.2.0 (Debian 1:6.2+dfsg-2ubuntu6.30)
Copyright (c) 2003-2021 Fabrice Bellard and the QEMU Project developers
eugene@Aurora:/mnt/c/Users/19902/zhilingji_experiment$ qemu-riscv64 --version
qemu-riscv64 version 6.2.0 (Debian 1:6.2+dfsg-2ubuntu6.30)
Copyright (c) 2003-2021 Fabrice Bellard and the QEMU Project developers
```

图 4.1: QEMU User 6.2.0 版本信息

5 ARM 程序设计

5.1 程序结构

ARM 程序由代码段和数据段两部分组成。代码段从 `_start` 标签开始执行，首先完成数组地址、数组长度、阈值、计数器和求和变量的初始化，然后进入循环处理过程。循环中每次读取一个数组元素，将其与阈值进行比较，并根据比较结果决定是否执行计数和累加操作。数据段中使用 `.word` 定义实验所需的整数数组。

ARM 程序的主要执行流程如下：

1. 使用 `ldr r0, =data` 将数组首地址保存到寄存器 `r0`；
2. 使用 `mov` 指令设置数组长度、阈值、计数器和求和变量；
3. 在循环中通过 `ldr r3, [r0], #4` 读取当前数组元素，并将数组地址后移；
4. 使用 `cmp r3, r2` 比较当前数据和阈值；
5. 使用带条件后缀的 `addge` 指令完成条件计数和条件累加；
6. 当所有数组元素处理完毕后，通过系统调用退出程序。

5.2 寄存器使用说明

ARM 程序中主要寄存器的分配如下表所示：

寄存器	作用
r0	保存数组当前访问地址
r1	保存剩余待处理的数据个数
r2	保存阈值 threshold
r3	保存当前读取的数组元素
r4	保存满足条件的数据个数 count
r5	保存满足条件的数据总和 sum
r7	保存 Linux 系统调用号

其中，`r0` 在每次读取数组元素后自动增加 4 字节，指向下一个整数；`r1` 用于控制循环次数；`r4` 和 `r5` 分别保存统计结果。程序运行结束时，ARM 版本中满足条件的数据个数 `count=4` 保存在 `r4` 中，满足条件的数据总和 `sum=58` 保存在 `r5` 中。由于本实验默认使用程序退出码展示求和结果，因此最终代码将 `r5` 的值移动到 `r0`，作为退出码返回。

5.3 核心代码说明

ARM 程序中最能体现本实验差异点的核心代码如下：

```
cmp     r3, r2
addge   r4, r4, #1
addge   r5, r5, r3
```

其中，`cmp r3, r2` 用于比较当前数据 `r3` 和阈值 `r2`。`addge` 中的 `ge` 表示 “greater or equal”，即“大于等于”。因此，当 `r3 >= r2` 时，`addge r4, r4, #1` 才会执行，用于完成 `count++`；`addge r5, r5, r3` 才会执行，用于完成 `sum += data[i]`。

这说明 ARM 在该条件统计场景中可以使用条件执行指令完成条件更新，不需要为该判断单独设置跳过统计代码的分支标签。

6 RISC-V 程序设计

6.1 程序结构

RISC-V 程序的整体结构与 ARM 程序相同，也包括初始化、循环处理、条件判断、统计更新和程序退出几个部分。程序首先将数组地址、数组长度和阈值加载到寄存器中，并将计数器和求和变量清零。随后进入循环，每次读取一个数组元素，并与阈值进行比较。

与 ARM 程序不同的是，RISC-V 程序没有使用条件执行指令，而是通过显式条件分支指令控制程序流程。当当前数据小于阈值时，程序跳转到 `skip` 标签，跳过计数和累加操作；当当前数据大于等于阈值时，程序不发生跳转，继续执行后面的统计指令。

RISC-V 程序的主要执行流程如下：

1. 使用 `la t0, data` 将数组首地址保存到寄存器 `t0`；
2. 使用 `li` 指令设置数组长度、阈值、计数器和求和变量；
3. 在循环中通过 `lw t3, 0(t0)` 读取当前数组元素；
4. 使用 `addi t0, t0, 4` 将数组地址后移；
5. 使用 `blt t3, t2, skip` 判断当前数据是否小于阈值；
6. 如果当前数据大于等于阈值，则继续执行计数和累加指令；
7. 所有数据处理完毕后，通过系统调用退出程序。

6.2 寄存器使用说明

RISC-V 程序中主要寄存器的分配如下表所示：

寄存器	作用
`t0`	保存数组当前访问地址
`t1`	保存剩余待处理的数据个数
`t2`	保存阈值 `threshold`
`t3`	保存当前读取的数组元素
`t4`	保存满足条件的数据个数 `count`
`t5`	保存满足条件的数据总和 `sum`
`a0`	保存程序退出码
`a7`	保存 Linux 系统调用号

其中，t0 用于保存数组访问地址，t1 用于控制循环次数，t2 保存阈值，t4 和 t5 分别保存计数结果和求和结果。程序运行结束时，RISC-V 版本中满足条件的数据个数 count=4 保存在 t4 中，满足条件的数据总和 sum=58 保存在 t5 中。由于本实验默认使用程序退出码展示求和结果，因此最终代码将 t5 的值移动到 a0，作为退出码返回。

6.3 核心代码说明

RISC-V 程序中最能体现本实验差异点的核心代码如下：

```
blt     t3, t2, skip
addi    t4, t4, 1
add     t5, t5, t3
```

其中，blt t3, t2, skip 表示如果当前数据 t3 小于阈值 t2，则跳转到 skip 标签。这样，程序会跳过后面的计数和累加指令。若当前数据大于等于阈值，则不发生跳转，继续执行 addi t4, t4, 1 和 add t5, t5, t3，分别完成计数和求和。

这说明 RISC-V 在该条件统计场景中主要通过显式条件分支改变程序控制流，从而实现与 ARM 条件执行指令相同的算法功能。

7 实验步骤

本节以实验指导的形式给出完整操作流程。读者应按照“编译、运行、反汇编、记录结果”的顺序完成实验，并重点观察核心循环中条件判断部分的实现方式。实验过程中不要求比较 ARM 与 RISC-V 的全部指令集差异，只需围绕条件执行与显式分支这一差异点展开分析即可。

7.1 编译 ARM 程序

首先进入实验文件所在目录。本实验中 ARM 源程序文件名为 arm.s，使用 arm-linux-gnueabi-as 将汇编源程序编译为目标文件，再使用 arm-linux-gnueabi-ld 链接生成可执行文件。

```
cd /mnt/c/Users/19902/zhilingji_experiment
arm-linux-gnueabi-as -o arm.o arm.s
arm-linux-gnueabi-ld -o arm.elf arm.o
```

其中，arm.o 为汇编后生成的目标文件，arm.elf 为链接后生成的 ARM 可执行文件。

7.2 运行 ARM 程序

ARM 程序使用 qemu-arm 运行。由于本实验为了简化输出过程，将最终求和结果作为程序退出码返回，因此运行程序后需要使用 echo $? 查看退出码。

```
qemu-arm ./arm.elf
echo $?
```

若程序运行正确，应得到如下结果：

```
58
```

该结果表示满足条件的数据之和为 58。

7.3 编译 RISC-V 程序

RISC-V 源程序文件名为 `riscv.s`。与 ARM 程序类似，先使用 `riscv64-linux-gnu-as` 进行汇编，再使用 `riscv64-linux-gnu-ld` 进行链接。

```
riscv64-linux-gnu-as -o riscv.o riscv.s
riscv64-linux-gnu-ld -o riscv.elf riscv.o
```

其中，`riscv.o` 为汇编后生成的目标文件，`riscv.elf` 为链接后生成的 RISC-V 可执行文件。

7.4 运行 RISC-V 程序

RISC-V 程序使用 `qemu-riscv64` 运行。同样地，程序将最终求和结果作为退出码返回，因此运行后使用 `echo $?` 查看结果。

```
qemu-riscv64 ./riscv.elf
echo $?
```

若程序运行正确，应得到如下结果：

```
58
```

ARM 与 RISC-V 两个程序均输出 58，说明它们完成了相同的数据分组统计功能。

7.5 验证计数结果

前面运行程序时，退出码返回的是满足条件数据的总和 `sum=58`。实际上，程序中也同时计算了满足条件的数据个数 `count=4`。ARM 版本将 `count` 保存在 `r4` 中，RISC-V 版本将 `count` 保存在 `t4` 中。

如果读者希望通过退出码直接验证 `count`，可以临时修改程序最后的返回寄存器。需要注意的是，该步骤只用于验证计数结果，提交的源代码仍建议保留默认返回 `sum` 的版本。

对于 ARM 程序，可以将结尾处：

```
mov r0, r5
```

临时改为：

```
mov r0, r4
```

然后重新编译并运行：

```
arm-linux-gnueabi-as -o arm.o arm.s
arm-linux-gnueabi-ld -o arm.elf arm.o
qemu-arm ./arm.elf
echo $?
```

此时应得到：

```
4
```

对于 RISC-V 程序，可以将结尾处：

```
mv a0, t5
```

临时改为：

```
mv a0, t4
```

然后重新编译并运行：

```
riscv64-linux-gnu-as -o riscv.o riscv.s
riscv64-linux-gnu-ld -o riscv.elf riscv.o
qemu-riscv64 ./riscv.elf
echo $?
```

此时应得到：

```
4
```

完成验证后，应将 ARM 程序恢复为 `mov r0, r5`，将 RISC-V 程序恢复为 `mv a0, t5`，使程序默认输出求和结果 58。

7.6 反汇编查看程序结构

为了观察两种指令集在核心循环中的实现差异，需要使用 `objdump` 对生成的可执行文件进行反汇编。由于完整反汇编内容较长，本实验只截取 `loop` 到 `done` 之间的核心循环部分进行观察。

查看 ARM 程序核心循环的命令如下：

```
arm-linux-gnueabi-objdump -d arm.elf | sed -n '/<loop>:/,/<done>:/p'
```

也可以将结果保存到文本文件中：

```
arm-linux-gnueabi-objdump -d arm.elf | sed -n '/<loop>:/,/<done>:/p' > arm_loop.txt
```

查看 RISC-V 程序核心循环的命令如下：

```
riscv64-linux-gnu-objdump -d riscv.elf | sed -n '/<loop>:/,/<done>:/p'
```

也可以将结果保存到文本文件中：

```
riscv64-linux-gnu-objdump -d riscv.elf | sed -n '/<loop>:/,/<done>:/p' >
    riscv_loop.txt
```

通过上述反汇编命令，可以直接观察两种程序的循环体结构以及条件判断部分的实现方式。

7.7 观察与记录重点

本实验观察的重点不是两个程序的普通初始化代码，而是循环体中对当前数据进行判断并决定是否统计的部分。具体来说，需要重点记录以下内容：

1. 默认运行时，两个程序返回的求和结果是否均为 `sum=58`；

2. 若进行可选验证，两个程序返回的计数结果是否均为 `count=4`；
3. ARM 程序核心循环中是否出现 `addge` 条件执行指令；
4. RISC-V 程序核心循环中是否出现 `blt` 条件分支指令和 `skip` 跳转标签；
5. 两种程序在实现 `if (data[i] >= threshold)` 时的代码结构有何不同；
6. 核心循环中指令条数是否存在差异。

其中，ARM 程序应重点观察如下代码：

```
cmp     r3, r2
addge   r4, r4, #1
addge   r5, r5, r3
```

RISC-V 程序应重点观察如下代码：

```
blt     t3, t2, skip
addi    t4, t4, 1
add     t5, t5, t3
```

如果读者正确完成上述实验步骤，应能发现：ARM 通过条件执行指令直接控制计数和累加操作是否执行，而 RISC-V 通过显式条件分支跳过统计代码。两者都能实现相同功能，但条件判断的控制方式不同。

8 预期结果与分析引导

按照前一节给出的步骤完成编译和运行后，读者应分别得到 ARM 和 RISC-V 两个程序的运行结果。两个程序使用相同的数据数组和阈值，实验数据为：

```
8, 12, 6, 20, 15, 9, 3, 11
```

阈值为 10，因此满足条件的数据为 12、20、15、11。理论上，满足条件的数据个数为 4，求和结果为 58：

```
count = 4
sum = 12 + 20 + 15 + 11 = 58
```

若 ARM 程序运行正确，应观察到如下命令结果：

```
qemu-arm ./arm.elf
echo $?
58
```

若 RISC-V 程序运行正确，应观察到如下命令结果：

```
qemu-riscv64 ./riscv.elf
echo $?
58
```

如果读者默认运行时得到的两个结果均为 58，则说明 ARM 和 RISC-V 两个版本都正确计算出了满足条件数据的总和。需要注意的是，程序同时也计算了满足条件的数据个数：ARM 版本的 `count=4` 保存在 `r4` 中，RISC-V 版本的 `count=4` 保存在 `t4` 中。若读者按照前文“可选：验证计数结果”的方法临时修改返回寄存器，也可以通过退出码观察到计数结果 4。

为了进一步观察两种指令集在实现方式上的差异，读者还需要对两个可执行文件进行反汇编，并重点查看 `loop` 到 `done` 之间的核心循环部分。

ARM 程序核心循环的反汇编结果如下：

```
00010088 <loop>:
   10088:       e3510000        cmp     r1, #0
   1008c:       0a000005        beq     100a8 <done>
   10090:       e4903004        ldr     r3, [r0], #4
   10094:       e1530002        cmp     r3, r2
   10098:       a2844001        addge   r4, r4, #1
   1009c:       a0855003        addge   r5, r5, r3
   100a0:       e2511001        subs    r1, r1, #1
   100a4:       eafffff7        b       10088 <loop>

000100a8 <done>:
```

其中，与条件统计直接相关的代码为：

```
cmp     r3, r2
addge   r4, r4, #1
addge   r5, r5, r3
```

这部分代码中，`cmp r3, r2` 先比较当前数据和阈值。随后两条 `addge` 指令只有在当前数据大于等于阈值时才执行，分别完成计数器加一和总和累加。也就是说，ARM 版本通过条件执行指令完成了条件统计操作。

RISC-V 程序核心循环的反汇编结果如下：

```
0000000000010100 <loop>:
   10100:       02030063                beqz    t1,10120 <done>
   10104:       0002ae03                lw      t3,0(t0)
   10108:       00428293                addi    t0,t0,4
   1010c:       007e4663                blt     t3,t2,10118 <skip>
   10110:       001e8e93                addi    t4,t4,1
   10114:       01cf0f33                add     t5,t5,t3

0000000000010118 <skip>:
   10118:       fff30313                addi    t1,t1,-1
   1011c:       fe5ff06f                j       10100 <loop>

0000000000010120 <done>:
```

其中，与条件统计直接相关的代码为：

```
blt     t3, t2, skip
addi    t4, t4, 1
```

```
3 add     t5, t5, t3
```

这部分代码中，`blt t3, t2, skip` 表示如果当前数据小于阈值，就跳转到 `skip` 标签，从而跳过后面的计数和累加指令。如果当前数据大于等于阈值，则不发生跳转，程序继续执行 `addi` 和 `add`，完成统计操作。

两种程序核心循环的指令数量对比如表所示：

程序版本	核心循环指令数	`if (data[i] >= threshold)` 的实现方式
ARM	8 条	使用 `cmp + addge + addge`。比较后直接让两条加法指令带条件执行，不需要额外的 `skip` 标签。
RISC-V	8 条	使用 `blt + skip`。当当前数据小于阈值时跳转到 `skip`，通过显式分支跳过统计代码。

从指令条数上看，本实验中 ARM 和 RISC-V 两个版本的核心循环均为 8 条指令，因此不能简单得出某一种指令集在该实验中代码条数更少的结论。本实验更主要的差异体现在条件判断后的控制方式上。

ARM 版本在比较当前数据和阈值后，使用 `addge` 这样的条件执行指令直接决定计数和累加操作是否生效。也就是说，ARM 将条件判断结果直接附加到后续指令上，使普通的加法指令具有了条件执行的效果。

RISC-V 版本则采用显式条件分支的方式实现相同逻辑。程序先通过 `blt` 判断当前数据是否小于阈值，如果条件成立就跳转到 `skip` 标签，跳过统计代码；如果条件不成立，则继续顺序执行计数和累加指令。

因此，本实验得到的主要结论是：在数据分组统计这一条件判断场景下，ARM 可以通过条件执行指令完成条件更新，而 RISC-V 通过显式分支指令改变程序控制流。两种方式都能实现相同的算法功能，但体现了不同的指令集设计思路。ARM 的条件执行方式使部分条件操作可以写得更紧凑，而 RISC-V 采用更规整、直接的分支结构，体现了其简洁、规则的 RISC 指令集设计特点。

9 实验流程设计

9.1 实验整体流程

本实验从数据分组统计问题出发，先设计算法，再分别编写 ARM 和 RISC-V 汇编程序。随后在 QEMU User 6.2.0 环境下完成编译和运行，并通过反汇编观察核心循环代码，最后对比两种指令集在条件判断实现方式上的差异。

实验整体流程如图 9.2 所示。



图 9.2: 实验整体流程

9.2 算法执行流程

本实验中的算法执行过程可以概括为：顺序遍历数组，将每个元素与阈值进行比较；如果当前元素大于等于阈值，则更新计数器并累加该元素；如果不满足条件，则跳过统计操作，继续处理下一个元素。所有数据处理完毕后，程序输出最终统计结果。

算法执行流程如图 9.3 所示。

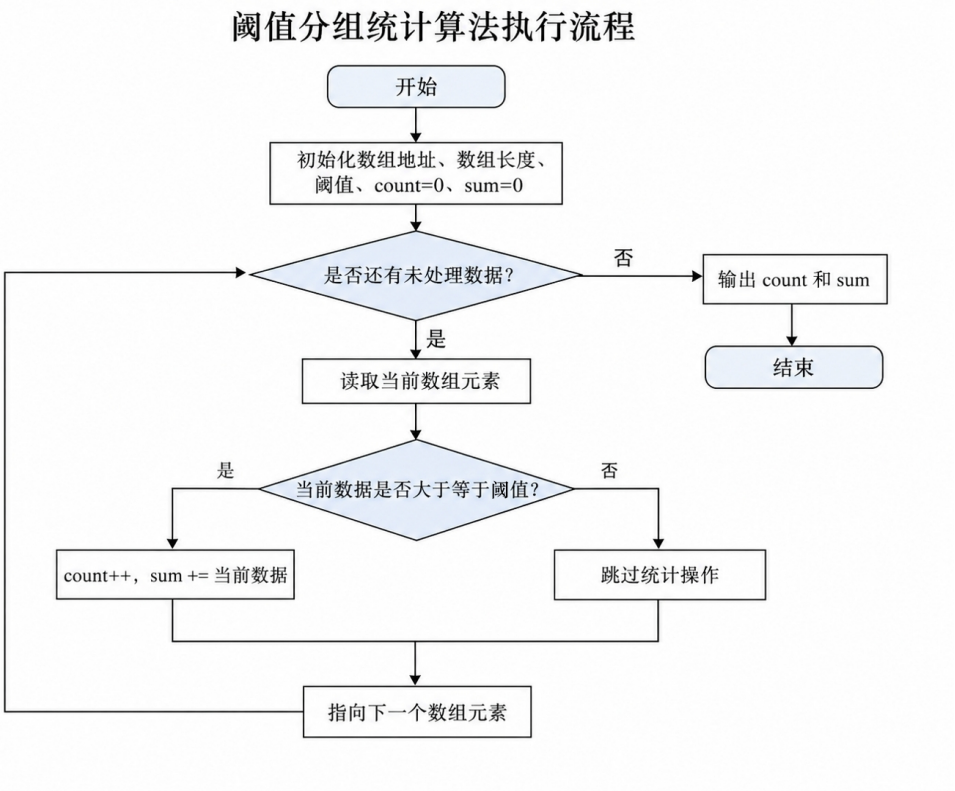


图 9.3: 阈值分组统计算法的执行流程

9.3 算法伪代码

为了更清晰地说明实验所实现的逻辑，下面给出本实验对应的算法伪代码。该伪代码与 ARM 和 RISC-V 两个版本的程序功能一致，均完成对数组中满足阈值条件数据的计数与求和。

Algorithm 1 基于阈值的数据分组统计算法

Input: 整数数组 `data`，数组长度 `n`，阈值 `threshold`
Output: 满足条件的数据个数 `count` 与总和 `sum`
`count` $\leftarrow 0$
`sum` $\leftarrow 0$
for $i \leftarrow 0$ **to** $n-1$ **do**
    if `data[i]` $\geq$ `threshold` **then**
        `count` $\leftarrow$ `count` $+1$
        `sum` $\leftarrow$ `sum` $+$ `data[i]`
    end if
end for
return `count`, `sum`

从伪代码可以看出，该算法的核心逻辑集中在循环内部的条件判断语句上，即：

```
if (data[i] >= threshold)
```

而本实验正是围绕这一核心逻辑，比较 ARM 与 RISC-V 两种指令集在汇编层面上的不同实现方式。

10 实验评价方法

为了判断读者是否真正完成并理解本实验，可以从实验操作、结果记录、代码理解和差异分析四个方面进行评价。评价重点不只在于程序是否能够运行，还在于读者是否能够从运行结果和反汇编代码中观察到 ARM 与 RISC-V 在条件判断实现方式上的差异。

具体评价标准如下表所示：

评价维度	分值	评价标准
实验操作完成情况	20 分	能够在 QEMU User 6.2.0 环境下正确编译并运行 ARM 和 RISC-V 两个汇编程序，能够得到正确的运行结果。
实验结果记录情况	20 分	能够记录 ARM 与 RISC-V 程序的运行命令、运行结果和反汇编命令，并说明两个程序默认返回的求和结果均为 `sum=58`；若进行可选验证，还应说明计数结果为 `count=4`。
代码理解情况	20 分	能够说明数组地址、循环计数器、阈值、当前数据、计数变量和求和变量分别存放在哪些寄存器中，并能解释核心循环的基本执行过程。
差异分析能力	30 分	能够明确指出本实验的核心差异是：同样实现 `if (data[i] >= threshold)`，ARM 使用 `cmp + addge + addge` 条件执行完成统计更新，RISC-V 使用 `blt + skip` 显式分支跳过统计代码，并能结合代码说明两种方式的区别。
总结表达情况	10 分	能够用较清楚的语言总结实验现象和指令集设计差异，结论与实验数据相符，表达较规范。

其中，差异分析是本实验评价的重点。如果读者只是运行程序得到结果，而不能结合核心循环代码说明 `addge` 和 `blt` 的作用，则说明其对实验目的的理解还不充分。较完整的分析应包括以下三点：第一，两个程序功能一致，均能得到求和结果 58；第二，ARM 通过条件执行指令直接完成条件计数和条件累加；第三，RISC-V 通过显式条件分支跳过不满足条件的数据处理代码。

因此，本实验的完成效果可以通过“能否运行、能否记录、能否解释、能否归纳”四个层次进行判断。只有当读者能够将运行结果、反汇编代码和指令集设计思想联系起来时，才说明其真正理解了本实验的核心内容。

11 实验总结

本实验围绕大数据处理中常见的阈值分组统计问题，设计了一个适合观察指令集差异的简单实验。读者需要分别使用 ARM 和 RISC-V 汇编程序实现同一算法：遍历数组 `8, 12, 6, 20, 15, 9, 3, 11`，筛选其中大于等于阈值 10 的数据，并计算其总和。按照实验步骤运行后，两个版本的程序都应得到结果 58，这说明两份汇编代码在功能上是一致的。

通过反汇编观察可以看到，两种程序在核心循环中都包含读取数据、比较阈值、更新统计结果和循环跳转等基本操作。但是，在条件判断后的实现方式上，二者存在明显差异。ARM 程序使用如下形

式的条件执行代码：

```
cmp     r3, r2
addge   r4, r4, #1
addge   r5, r5, r3
```

其中，addge 指令只有在比较结果满足“大于等于”时才会执行，因此可以直接完成条件计数和条件累加。RISC-V 程序则使用如下形式的显式分支代码：

```
blt     t3, t2, skip
addi    t4, t4, 1
add     t5, t5, t3
```

其中，blt 指令用于判断当前数据是否小于阈值。如果条件成立，程序跳转到 skip 标签，跳过后续统计操作；如果条件不成立，则继续执行计数和求和指令。

完成本实验后，读者应能够明确说明本实验的核心差异：对于同一个 if (data[i] >= threshold) 条件判断，ARM 版本使用 cmp + addge + addge，将比较结果直接附加到后续加法指令上，从而减少显式跳转；RISC-V 版本使用 blt + skip，通过条件分支改变控制流，在数据不满足条件时跳过统计代码。本实验中两种程序的核心循环指令数均为 8 条，因此不能简单认为某一种指令集在该任务中指令条数更少。更准确地说，本实验体现的是两种指令集在条件判断表达方式和控制流组织方式上的差异。

从指令集设计角度看，ARM 的条件执行机制增强了部分指令的表达能力，使一些简单条件操作可以直接写成带条件后缀的指令；RISC-V 则保持指令格式和控制逻辑的简洁，通过显式分支完成条件控制。这种差异体现了不同 RISC 指令集在功能表达能力和指令规整性之间的设计取舍。读者在完成本实验后，应能够结合具体汇编代码说明这一差异，而不是只停留在运行结果层面。